

COSC 214 – Data Structures and Algorithms

Semester Project: Evaluating Data Structures Using a Healthcare Dataset

1. Project Overview

Modern software systems frequently need to store, retrieve, and process large volumes of structured data efficiently. The **choice of data structure** plays a critical role in determining the performance of these systems.

In this semester project, you will explore how different data structures perform when storing and managing structured records derived from a healthcare-style dataset. The primary goal is to apply the concepts learned in **COSC 214 (Data Structures and Algorithms)** to a realistic data management problem.

Students will:

- Design a data model for healthcare-style records
- Implement multiple data structures
- Perform core operations (insert, search, delete, traverse)
- Conduct performance benchmarking
- Analyze how different structures impact algorithm efficiency

This project emphasizes **algorithmic thinking, implementation skills, and empirical performance evaluation**.

2. Dataset Used in the Project

Dataset Source:

<https://www.kaggle.com/datasets/prasad22/healthcare-dataset>

This dataset contains approximately **10,000 synthetic healthcare records**. It is designed for educational purposes and does not contain real patient data.

Example fields in the dataset include:

- Name
- Age
- Gender
- Blood Type
- Medical Condition
- Date of Admission
- Hospital
- Doctor
- Insurance Provider
- Billing Amount
- Room Number
- Admission Type
- Discharge Date
- Medication
- Test Results

Students **do not need to use all columns**. Instead, the project will focus on a simplified record structure.

3. Simplified Record Model

Students will convert each dataset row into a **PatientRecord** object.

Example structure:

PatientRecord

- ID
- Age
- Gender
- MedicalCondition
- Hospital
- AdmissionType
- BillingAmount

Students should generate a unique ID if one is not provided.

Example:

```
class PatientRecord {  
    int id;  
    int age;
```

```
String gender;  
  
String medicalCondition;  
  
String hospital;  
  
String admissionType;  
  
double billingAmount;  
  
}
```

This simplified structure ensures that the focus remains on **data structures rather than dataset complexity**.

4. Data Structures to Implement

To keep the project focused and manageable, students will implement the following four structures:

1. **Dynamic Array (ArrayList style structure)**
2. **Linked List**
3. **Hash-Based Structure (HashSet or HashMap)**
4. **Queue or Priority Queue**

Students may implement these **from scratch OR use built-in language libraries**, depending on instructor preference.

Recommended approach:

Use built-in structures for storage but focus on **algorithm analysis and benchmarking**.

5. Required Operations

Each data structure must support the following operations on PatientRecord objects:

Insert

Add a new patient record.

Search

Search for a patient record using the **record ID**.

Delete

Remove a record by ID.

Traversal

Iterate through all records.

Students should implement wrapper functions such as:

insertRecord()

searchRecord()

deleteRecord()

traverseRecords()

6. Queue / Priority Queue Use Case

The queue structure will simulate **patient admission processing**.

Example use:

Patients are processed in order of admission.

Priority queues can simulate **emergency vs normal admissions**.

Example priority rule:

Emergency patients have higher priority than regular admissions.

This demonstrates how queues are useful for **task scheduling systems**.

7. Project Timeline (8 Weeks)

Total Duration: **8 weeks**

Weeks 1–3: Design Phase ----- Part 1

Students will:

- Explore the dataset
- Define the PatientRecord structure
- Design how each data structure will store records
- Define algorithms for insert/search/delete/traverse

Deliverable:

Design Document

The design document should include:

- Description of the PatientRecord structure
- Explanation of each data structure
- Pseudocode for operations
- Basic system architecture diagram (optional but recommended)

Expected length: **2–3 pages**.

Weeks 4–8: Implementation Phase ----- Part 2

Students will:

- Load the dataset
- Implement required data structures
- Implement record operations
- Run benchmarking experiments
- Analyze results
- Prepare presentation

Recommended milestones:

Week 4 – dataset loading complete

Week 5 – two data structures implemented

Week 6 – all structures implemented

Week 7 – benchmarking completed

Week 8 – final report and presentation

8. Dataset Loading Instructions

Students must load the Kaggle dataset CSV file and convert rows into PatientRecord objects.

Students may use:

- Java file readers
- Python CSV library
- pandas (optional)

Example workflow:

1. Load CSV file
2. Extract required columns
3. Create PatientRecord objects
4. Insert records into data structures

Students **do not need to clean the dataset extensively**.

9. Performance Benchmarking

Students will measure performance of operations using datasets of different sizes.

Suggested experiment sizes:

100 records

1,000 records

5,000 records

10,000 records

Operations to measure:

- Insert time
- Search time
- Delete time

Benchmarking method:

Students should run each experiment **at least 5 times and compute the average runtime and memory.**

Timing tools:

Java: `System.nanoTime()`

Python: `time` module

Example result table:

Data Structure	Insert Time	Search Time	Delete Time
Array			
Linked List			
Hash Structure			
Queue / Priority Queue			

Students should interpret results and relate them to **Big-O time complexity.**

10. Expected Deliverables

Students must submit the following:

1. Source Code

Implementation of the required data structures and operations.

2. Design Document

2–3 pages describing system design.

3. Performance Analysis Report

Includes:

- experiment methodology
- benchmark tables
- performance discussion

Expected length: **3–5 pages**.

4. Presentation

Each team will give a **8–10 minute presentation**.

Presentation should include:

- problem overview
- data structures implemented
- benchmarking results
- lessons learned

All team members must present.

11. Team Structure

Teams consist of **4 students**.

Implementation responsibilities will be divided so that individual performance can be measured.

Example division:

Student 1

ArrayList implementation

Student 2

Linked List implementation

Student 3

Hash structure implementation

Student 4

Queue / Priority Queue implementation + benchmarking framework

All members collaborate on design, report writing, and presentation.

12. Skills Developed

Students will develop:

Technical Skills

- Data structure implementation
- Algorithm performance analysis
- Benchmarking experiments
- Data processing from real datasets

Analytical Skills

- Comparing algorithm efficiency
- Understanding tradeoffs between data structures

Communication Skills

- Writing technical reports
- Presenting technical findings

13. Grading Rubric (Total: 8 Points)

Component	Points	Description
Design	2	Clear design of the PatientRecord model and explanation of how each data structure stores and manages records.
Implementation	2	Correct implementation of the required data structures and operations (insert, search, delete, traverse). Implementation points are divided among three team members (~0.67 each) to evaluate individual contributions.
Performance Analysis	2	Benchmark experiments conducted correctly with results presented and analyzed. Students should compare performance across data structures and relate findings to algorithm complexity.

Presentation	2	Clear and organized presentation explaining the project design, implementation approach, benchmarking results, and key conclusions.
--------------	---	---

14. Project Goal

The goal of this project is to demonstrate how **different data structures impact the efficiency of real-world data processing tasks**.

By implementing and benchmarking multiple structures, students will gain practical insight into why **algorithm and data structure selection is critical in software system design**.